Conference Paper Title*

*Note: Sub-titles are not captured in Xplore and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Open model hubs have turned pretrained checkpoints into reusable software artifacts. A developer can now download a model from repositories such as Hugging Face and execute it with only a few lines of code [**?**]. This convenience also creates a new supply-chain attack surface: a model file may embed hidden behavior that is triggered during deserialization or inference, even when the surrounding project appears benign. Unlike ordinary application code, model artifacts are opaque, framework-dependent, and routinely trusted by downstream users, which makes pre-deployment inspection particularly difficult.

Recent studies show that insecure serialization remains widespread on public model hubs, and that malicious model artifacts can be published and propagated through ordinary sharing workflows [**?**]. Existing defenses mainly rely on *static* inspection: they scan serialized files, decompile pickles, or search for suspicious APIs and import patterns [**?**], [**?**], [**?**]. These tools are useful as a first barrier, but they reason about code structure rather than executed behavior. As a result, they often over-flag benign utilities while still missing attacks hidden behind legitimate APIs, deferred execution, or code obfuscation. This limitation becomes particularly severe for camouflage attacks such as TensorAbuse, where an attacker repurposes authorized TensorFlow interfaces to perform file access, network exfiltration, code injection, or shell acquisition without obviously malicious syntax [**?**].

Identify applicable funding agency here. If none, delete this.

Therefore, it is more effective to focus on whether the model performs malicious operations at runtime. However, establishing a robust runtime defense for model artifacts remains nontrivial. A fundamental limitation of existing dynamic detection methods is their strict reliance on a priori knowledge. For instance, current instrumentation techniques typically require predefined attack signatures or expert-guided heuristics to determine which specific functions to hook, while system call monitors heavily depend on static whitelists to filter out benign events. Because adversarial techniques iterate rapidly, such defenses relying on prior knowledge possess inherent vulnerability. They not only suffer from high rates of false positives and negatives, but they are also fundamentally defenseless against zero-day exploits that lack established signatures.

To address the inherent limitations of such defense strategies, we shift our focus from hunting known threats to modeling benign behavioral characteristics. Our starting observation is that benign AI models exhibit highly regular runtime templates. These execution phases are dictated by the framework's internal logic and remain largely invariant regardless of the model's specific architecture or task. Normal execution consistently passes through a small set of stable phases, including deserialization, tensor materialization, and library loading. In contrast, malicious models inevitably deviate from these strict templates, exhibiting anomalous patterns such as unexpected file access during deserialization or unauthorized network communication during inference.

Building on this observation, we propose a defense mechanism that requires no prior knowledge. We first employ an unsupervised, statistical approach to screen for deviations from these regular benign templates. To capture the necessary context without imposing filtering biases, we implement a lightweight, universal instrumentation strategy that hooks all library API calls, completely eliminating the need for fragile whitelist filtering. By combining statistical baseline

profiling with comprehensive, low-overhead monitoring, our approach effectively neutralizes both known and zero-day threats without requiring any prior knowledge of specific attack methodologies.

Specifically, MAD leverages this regularity to turn dynamic model security into a two-stage behavior analysis problem. In the first stage, an unsupervised anomaly detector learns the distribution of benign operation patterns and efficiently filters out normal activity. In the second stage, only statistically abnormal operations are escalated for cross-layer analysis, where they are associated with high-level Python semantic context collected via adaptive instrumentation. This association bridges the semantic gap between low-level system events and model intent, enabling LLM-based reasoning to determine whether the observed behavior is consistent with benign execution or indicative of an attack. Through this two-stage pipeline, MAD achieves both high detection accuracy and low false positive rates while maintaining practical deployability.

With these designs, MAD achieves superior detection performance and operational efficiency. Experiments demonstrate superior qualitative and quantitative results. In summary, our contributions are as follows:

- We propose the first hybrid dynamic detection system for AI model supply chain attacks that combines non-bypassable system-call evidence with high-dimensional Python semantic context, bridging the semantic gap between low-level behaviors and model intent.
- We develop a robust cross-layer association mechanism that aligns abnormal system-level operations with high-level semantic events, enabling LLM-based reasoning to provide accurate, explainable, and auditable security alerts.
- Extensive experiments on real-world and synthetic malicious models demonstrate that our system achieves high detection accuracy and low false positive rates, outperforming existing static and dynamic defenses while maintaining practical deployability.

II. Background

A. Malicious Model Artifacts

The rapid advancement of machine learning and the proliferation of open-source model hubs have fundamentally transformed how AI models are developed and deployed. Developers increasingly upload their pre-trained models to centralized platforms such as Hugging Face and TensorFlow Hub , enabling the broader community to freely download, fine-tune, and integrate these artifacts into downstream applications. This collaborative ecosystem forms the core of the AI model supply chain. According to Hugging Face statistics, as of 2026, the platform hosts over 2.8 million models, supporting more than 1,000 languages and dozens of distinct tasks. Foundational models like BERT and GPT have become indispensable tools for developers, each recording over ten million downloads. This widespread adoption underscores the critical role that model sharing—and by extension, the model supply chain—plays in accelerating both academic research and industrial technological progress.

While this paradigm of model sharing fosters immense collaborative innovation, it simultaneously introduces severe security vulnerabilities within the AI model supply chain. Recent security audits have increasingly exposed structural flaws in platforms like Hugging Face. Consequently, a surge of real-world malicious attacks targeting this supply chain has been reported. In these attacks, malicious actors upload compromised models designed to execute unauthorized actions when downloaded and deployed by unsuspecting users. These supply chain compromises are not merely theoretical; they have already resulted in the large-scale theft of user credentials and the illicit exploitation of Large Language Model API tokens, causing significant financial and operational damage to the affected organizations.

The primary reason the AI model supply chain is highly susceptible to these attacks lies in the stealthy nature of malicious code insertion and the inherent difficulty in its detection. Unlike traditional software, AI models are often distributed as opaque, binary artifacts. Attackers exploit this by embedding malicious payloads directly into the model structure, using various obfuscation techniques. For instance, malicious code can be intricately disguised as normal model components or hidden within the unreadable binary formats of the model files to successfully evade the static security scanners employed by model hosting platforms.

Recent studies have provided comprehensive analyses of these malware poisoning attacks within pre-trained model hubs. Particular attention has been drawn to the exploitation of insecure serialization formats, which allow attackers to embed arbitrary execution commands that trigger automatically upon model loading. Because these payloads execute before the model even begins inference, and are woven into the legitimate deserialization process, they easily bypass conventional, surface-level detection mechanisms, making defense a formidable challenge.

B. Existing Defenses and Their Limitations

While extensive research has focused on inference-time threats—such as adversarial examples and data poisoning—efforts to secure the AI model supply chain against pre-execution malware injection remain comparatively limited and simplistic. Current defensive strategies primarily rely on static analysis and signature-based scanning tools to inspect model artifacts before deployment. For instance, Fickling was developed as a decompiler, static analyzer, and bytecode rewriter specifically for Python's pickle object serialization. By decompiling the opaque pickle data stream into readable Python code, Fickling attempts to identify embedded malicious payloads. Similarly, the Hugging Face platform employs a custom security scanner, Picklescan, which combines the traditional antivirus engine ClamAV with a mechanism to extract and inspect the import tables within pickle files. Furthermore, in tandem with the disclosure of the TensorAbuse attack, a detection methodology based on statically searching for sus-

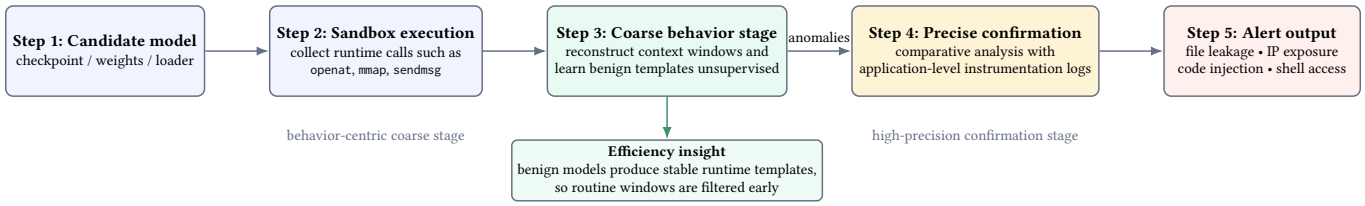

Fig. 1: Workflow of MAD.

picious API calls within computational graphs was proposed. The foremost limitation of these tools is their heavy reliance on static pattern matching—specifically, detecting the presence of predefined libraries or suspicious function calls. Because they do not analyze the actual dynamic execution of the code, they fail to establish semantic context. A benign model component fetching remote weights and a malicious payload establishing a reverse shell might both utilize the same network-related library. This lack of runtime semantic analysis inevitably leads to high rates of both false positives and false negatives.

On the other hand, because these works generally lack a comprehensive understanding of the evolving attack vectors within the AI model supply chain, their designs are inherently reactive. They are typically engineered to address specific, previously observed vulnerabilities rather than adopting a holistic approach to model behavior. This strategy severely restricts their ability to defend against zero-day exploits or novel attack methodologies that do not conform to known signatures.

III. Design

In this section, we present the design of our hybrid defense system for detecting malicious behaviors embedded in model artifacts. We first formalize the problem setting and design goals of the system. Then we provide a high-level system overview and detail three core modules that jointly model (i) non-bypassable low-level system-call evidence and (ii) high-dimensional Python semantic context collected via non-invasive instrumentation. Finally, we discuss deployment considerations that enable practical and safe use in real-world analysis environments.

A. System Overview

Input. The system takes a model artifact M (e.g., a checkpoint/weights file, a serialized object, or a model package) and a controlled execution configuration E.

Observations. During execution, we collect two synchronized streams: an application-layer semantic log stream $L = \{(\ell_i, t_i)\}$ from Python instrumentation, and a kernel-visible system-call stream $S = \{(s_j, \tau_j)\}$ from system-call tracing.

Output. The system outputs a structured alert

$$A = \langle \textit{IntentScore},\ \textit{ThreatCategory},\ \textit{Context} \rangle, \qquad (1)$$

where *IntentScore* $\in [0, 1]$ reflects the likelihood of malicious intent, *ThreatCategory* is the predicted attack type, and *Context* is an evidence chain linking semantic context to low-level behaviors, which contains abnormal system-call operations and aligned high-level Python semantic events.

Goals. Our design targets three goals: (G1) *non-bypassability* via system-call evidence; (G2) *low false positives and explainability* via semantic context; and (G3) *efficiency and deployability* via fast prefiltering and lightweight instrumentation.

Module 1: Kernel-level behavior extraction and prefiltering. We aggregate raw system calls into higher-level "operations" and use an Autoencoder to quickly identify abnormal operations. This module turns noisy, fine-grained traces into compact operation vectors and filters out clearly benign behavior, ensuring that only suspicious low-level activities are forwarded for deeper analysis.

Module 2: Application-layer semantic instrumentation. We perform non-invasive, framework-adaptive Python instrumentation to capture high-dimensional semantic context around security-relevant API calls. This module logs which libraries and APIs are invoked, with what summarized arguments and in which execution phase, providing the semantic side of the evidence chain.

Module 3: Cross-layer association and LLM reasoning. We align abnormal low-level operations with nearby semantic contexts, then ask an LLM to judge whether the behavior is explainable under benign model execution, producing an intent score and an evidence chain. This module links what the model code appears to be doing with what the operating system actually observes, and converts the combined evidence into a human-understandable alert.

B. Kernel-level Behavior Extraction and Prefiltering

The second module operates at the operating system level and is responsible for turning raw system-call traces into a small set of suspicious "operations" for further cross-layer analysis. Directly modeling single system calls is impractical because they are extremely fine-grained and voluminous, and because meaningful behavior always manifests as a *sequence* of related calls.

System-call collection and filtering. During controlled model execution, we use `strace` to continuously trace system calls of the target process. To reduce noise and overhead, we immediately discard high-frequency but semantically uninformative calls such as scheduler hints. The selection of system calls focuses on those that are security-relevant, ensuring that the tracing process captures meaningful behaviors while minimizing unnecessary data collection. Table I summarizes the security-relevant syscall families we monitor and their

TABLE I: Security-relevant system-call categories used by our filter.

Filter param	Category	Purpose
file	File operations (`read`/`write`)	Detect sensitive data leakage or configuration file tampering
network	Network operations (`socket`)	Detect data exfiltration or remote command execution
cred	Credential operations (`setuid`)	Detect privilege-escalation behavior
desc	Descriptor operations (`dup`)	Assist file-related anomaly detection
signal	Signal handling (`kill`)	Capture malicious process termination attempts
clock	Time-related operations	Assist temporal anomaly analysis (e.g., abnormal delays)

roles in our detection pipeline. For example, file operations can indicate attempts to access sensitive data or modify configurations, while network operations may signal data exfiltration or remote command execution.

We categorize the selected system calls into two types: (1) *Directly malicious calls*, such as `execve` and `chmod`, which are directly converted into operation vectors $\mathbf{v}_{dir}$; and (2) *Stateful calls*, such as `open`, `read`, `write`, and `socket`, which require aggregation over their lifecycle.

Directly malicious calls $\mathbf{v}_{dir}$ are embedded using a pre-trained Bert model. Specifically, each syscall is encoded into a high-dimensional vector representation, capturing its semantic meaning. These embeddings are then resized or reduced to a fixed dimension d.

Stateful aggregation. Let $S = \{(s_k, \tau_k)\}_{k=1}^{N}$ denote the traced system-call stream, where s_k is the k-th syscall and τ_k its timestamp. For stateful operations, we maintain per-resource state keyed by $r = \langle PID, FD \rangle$, where *FD* denotes a file or socket descriptor. For each key r, we collect the indices of calls that operate on this resource within one lifecycle segment,

$$I_r = \{\, k \mid (s_k, \tau_k) \in S,\ s_k \text{ uses resource } r,\ \tau^r_{start} \le \tau_k \le \tau^r_{end} \,\}, \tag{2}$$

where τ^r_{start} and τ^r_{end} are the timestamps of the resource-acquisition and resource-release calls (e.g., `open` / `close` or `socket` / `shutdown`). The lifecycle segment for resource r is then the ordered subsequence $S_r = \{(s_k, \tau_k)\}_{k \in I_r}$.

We aggregate this segment into a fixed-length operation vector by applying a feature-extraction function ϕ over all calls in the segment:

$$\mathbf{v}_{sys}(r) = \phi(S_r) \in \mathbb{R}^d, \tag{3}$$

where ϕ computes lightweight statistics such as per-call-type counts, total bytes read and written, error-code distributions, and the duration $\tau^r_{end} - \tau^r_{start}$. Intuitively, each $\mathbf{v}_{sys}(r)$ summarizes "what happened to this particular file or connection" during its lifetime, smoothing out scheduling artifacts and interleaving from other threads.

The final feature matrix combines $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$, enabling the system to capture both direct and aggregated syscall behaviors. This hybrid representation ensures that directly malicious actions are immediately flagged, while stateful operations are analyzed in context to detect more subtle threats.

(SF: also formalize this part. I think this part can be merged with the previous one.)

Autoencoder-based anomaly detection. To automatically distinguish normal and abnormal operations without relying on handcrafted signatures, we train an Autoencoder on operation vectors collected from clean model loading and inference runs. Given an operation vector $\mathbf{v}$, the Autoencoder computes a reconstruction error

$$E = \|\mathbf{v} - \mathit{Dec}(\mathit{Enc}(\mathbf{v}))\|_2^2, \tag{4}$$

where *Enc* and *Dec* denote the encoder and decoder networks, respectively. Operations with reconstruction error E below a threshold θ are treated as consistent with normal model behavior and are discarded. If $E > \theta$, we mark the corresponding operation as abnormal, retain its full underlying system-call segment S' and metadata $\langle PID, TID, t_{start}, t_{end} \rangle$, and promote it to Module 3 for semantic association and intent reasoning.

By performing this unsupervised prefiltering at the operation level, the system keeps the expensive cross-layer association and LLM reasoning off the critical path for the vast majority of benign activity, while still surfacing rare, suspicious low-level behaviors that deviate from the learned distribution of normal AI model execution.

C. Non-invasive Python Semantic Instrumentation

The goal of this module is to collect high-dimensional Python semantic context that explains why a model issues certain low-level operations. The resulting semantic log stream is later correlated with system calls to bridge the "semantic gap" between model-level intent and kernel-level behavior.

Data Collection Each semantic event records process identifiers, a timestamp, the invoked function name and a compact argument summary:

$$\ell_i = \langle PID,\ TID,\ FuncName,\ ArgsSummary \rangle. \tag{5}$$

where *FuncName* typically corresponds to security-relevant framework or standard-library APIs. *ArgsSummary* aggregates only lightweight, non-sensitive information such as tensor shapes and dtypes, configuration flags, truncated or hashed file paths, and bounded-length string summaries. In addition to the immediate call, we optionally attach lightweight context so that downstream modules can reconstruct an interpretable execution narrative.

LLM-assisted Instrumentation. To achieve robust and adaptive monitoring across rapidly evolving AI frameworks, we propose an automated, dynamic API discovery and patching mechanism rather than relying on statically hardcoded function lists. Directly instrumenting heterogeneous frameworks is inherently challenging because their internal abstractions and public APIs change frequently. To avoid the prohibitive overhead of modifying framework source code or maintaining exhaustive API checklists, we leverage Large Language Models to synthesize dynamic probe generation scripts. Rather than manually specifying which APIs to hook, our methodology

employs runtime monkey patching. The LLM is guided to generate code that iterates over the target module's namespace at process startup, systematically identifying all public callable attributes while bypassing private or internal methods.(SF: this part need more details. what's your prompt? how do you select the API? how you automate the instrumentation?)

Algorithm 1: Automated Dynamic Instrumentation Logic

Input: Target library module $\mathcal{M}$ (e.g., `torch`, `tensorflow`)
Output: Instrumented module capable of context logging

Procedure `ApplyDynamicPatches`($\mathcal{M}$)**:**
  foreach *attribute name* $attr\_name$ **in** dir($\mathcal{M}$) **do**
    if $attr\_name$ *starts with "_"* **then**
      continue
    $target\_func \leftarrow$ getattr($\mathcal{M}, attr\_name$);
    if not *is_callable*($target\_func$) **then**
      continue
    $wrapper \leftarrow$ `Wrapper`($target\_func, attr\_name$);
    try
      setattr($\mathcal{M}, attr\_name, wrapper$);
    catch *ReadOnlyAttributeError*
      continue

Procedure `Wrapper`($func, name$)**:**
  return a closure that captures the full `traceback`, logs to JSON, executes $func$, and delegates return value;

The core of our approach lies in a rigorously structured prompt template that constraints the LLM to synthesize comprehensive and safe semantic wrappers. Specifically, the prompt defines a clear security objective—detecting anomalous activities during model loading and execution—and imposes strict technical constraints to ensure operational stability. The LLM is instructed to construct wrappers that capture highly detailed execution contexts before and after the original function invocation. This context includes timestamps, operation types, argument values, return types, and the complete call stack retrieved via the `traceback` module. To facilitate automated downstream analysis, the prompt mandates that all intercepted semantic events ℓ_i must be serialized into a standardized JSON format. The automated instrumentation logic synthesized by our prompt is abstracted in Algorithm 1.

The instrumentation is designed to be adaptive: when the underlying framework version changes, we only need to regenerate the wrappers for the new API surface, without changing the core detection pipeline. This enables broad coverage across diverse model frameworks while keeping manual engineering effort low.

Safety constraints. To reduce the risk of breaking runtime behavior, the instrumentation follows a conservative policy. First, we only wrap callable functions and avoid replacing class definitions, built-in data types, or objects backed by low-level C/C++ bindings. Second, wrappers are kept as thin as possible: they perform best-effort argument summarization and then immediately invoke the original function, without altering return values or side effects. Third, the parameter serialization adopts defensive strategies to control logging overhead by recording only shapes instead of raw tensor contents, truncating long strings, and other similar methods.

Furthermore, to preserve the original control and data flow of the ML framework, the generated script enforces graceful exception handling. Modern AI frameworks typically possess highly complex internal architectures, heavily utilizing read-only properties, immutable C-extension bindings, and dynamically generated descriptors. A naive monkey-patching approach would inevitably trigger fatal runtime errors when attempting to overwrite these restricted components, thereby crashing the process during model initialization. To mitigate this risk, our framework's prompt design constraints the LLM to wrap both the dependency injection phaseand the runtime execution phase of the generated wrappers within protective `try-except` blocks.

D. Cross-layer Spatio-temporal Association

The third module receives abnormal operations reported by Module 1 and associates them with high-dimensional Python semantic context from Module 2. Its primary goal is to determine whether each abnormal low-level operation can be reasonably explained as benign model behavior; only for operations that remain suspicious do we further construct a coherent cross-layer behavior profile that explains which model code triggered them and why they may be malicious.

System calls and Python semantics are inherently asynchronous due to operation system scheduling and concurrent I/O. As a result, naive one-to-one timestamp matching between logs S and L is unreliable in high-concurrency workloads. We therefore adopt a fuzzy spatio-temporal association strategy.

For each abnormal operation, Module 1 provides the underlying system-call segment S' and its metadata $\langle PID, TID, t_{start}, t_{end} \rangle$. Given this metadata and the semantic log stream $L = \{(\ell_i, t_i)\}$, Module 3 first retrieves candidate semantic events that could have triggered the operation. Concretely, we treat $\langle PID, TID \rangle$ as a primary key and restrict attention to semantic events from the same process.

Temporal windowing. We then apply a slack temporal window around the abnormal operation. For each candidate abnormal operation, we define its active time span $[t_{start}, t_{end}]$ based on the first and last system call in S', and search semantic events whose timestamps t_i fall into an expanded window $[t_{start} - \Delta t, t_{end} + \Delta t]$. The slack parameter Δt tolerates minor scheduling jitter while still localizing semantic context to the period when the resource was actually in use.

Spatial hints. Temporal proximity alone is insufficient under heavy concurrency. We therefore further filter candidates using resource-related hints derived from both layers, such as whether the operation is file- or network-related, file paths or prefixes, remote destinations, and other descriptor metadata

when available. Combining temporal and spatial constraints yields a fuzzy but robust alignment that significantly reduces spurious associations when many model components execute in parallel.

Cross-layer behavior profile. After spatio-temporal filtering, Module 3 bundles the abnormal operation vector, its raw system-call summary, and the aligned semantic events into a structured cross-layer behavior profile. This profile serves as the input to the LLM-based intent reasoning stage: it contains (i) what the OS observed at the system-call level, (ii) which high-level Python APIs and call sites were active around that time and on the same resource, and (iii) basic execution-phase information. Together, these elements provide sufficient context to decide whether the abnormal low-level behavior is consistent with benign model execution or indicative of malicious intent.

E. LLM-based Intent Reasoning and Evidence Chain Generation

Given an abnormal low-level operation and its aligned semantic contexts, the LLM decides whether the behavior is consistent with benign model execution.

Prompt construction. We translate the aligned evidence into a structured natural-language prompt that includes: (i) a concise summary of the abnormal operation (operation type, key statistics, and risk-relevant parameters), (ii) the aligned semantic call contexts (library/API, call stack summary, and argument summaries), and (iii) the execution phase.

Decision and explanation. The LLM outputs *IntentScore*, a *ThreatCategory* label, and a rationale. Importantly, we persist the subset of semantic events and system-call summaries used in the reasoning as the *evidence chain Context* for auditing.

Fail-safe behavior. If high-risk system-call patterns appear without any plausible semantic explanation (e.g., sensitive network connection with no corresponding high-level network API context), the system raises a high-severity alert and can optionally block execution in the controlled environment.

F. Deployment Considerations

Controlled execution. The design assumes model analysis runs in a controlled environment with least privilege and optional network isolation, so that confirmed malicious behaviors can be contained.

Extensibility. The instrumentation templates can be expanded as new framework APIs emerge; similarly, operation features can be extended to cover new low-level behaviors without changing the overall pipeline.

IV. Evaluation

We evaluate the effectiveness of MAD in detecting malicious behavior embedded in model artifacts. Our study focuses on three aspects: (i) detection performance compared to state-of-the-art defenses, including robustness to camouflage attacks such as TensorAbuse; (ii) the impact of cross-layer modeling and its components on accuracy and explainability; and (iii) runtime overhead and deployability in realistic analysis environments. We organize the evaluation around the following research questions:

- **RQ1:** Can MAD accurately detect malicious model artifacts.
- **RQ2:** Can MAD distinguish camouflaged, stealthy malicious models from benign models?
- **RQ3:** What is the runtime overhead of MAD, and is it practical to deploy in real-world model workflows?
- **RQ4:** How does each part of MAD contribute to the detection performance?

A. Experiment Setup

In this section, we describe the datasets, baselines, and implementation details used in our evaluation.

Datasets. We evaluate MAD on two distinct datasets to comprehensively assess its detection capabilities against both known, rudimentary vulnerabilities and complex, advanced supply-chain attacks:

- *In-the-wild Malicious Models.* The first dataset consists of known malicious models identified by prior research using MalHug. Specifically, this includes 87 models exploiting Pickle serialization vulnerabilities and 15 models abusing Keras custom Lambda layers to execute malicious behaviors. While these models represent real-world threats, their attack methodologies are relatively monolithic, rely on well-known deserialization flaws, and are generally straightforward to detect using existing static or signature-based scanning tools.
- *Advanced TensorAbuse Synthetic Corpus.* To address the limitations of the first dataset and evaluate our system against highly stealthy, framework-native attacks, we constructed a second, more challenging dataset using TensorAbuse. We selected 10 diverse foundation models spanning various domains to serve as benign baselines. We then systematically embedded malicious payloads directly into the computational graphs of these models. Specifically, we designed four categories of advanced attacks: File Exfiltration, IP Exposure, Malicious Code Insertion, and Remote Shell Acquisition. For each category, we implemented two distinct attack methodologies utilizing TensorAbuse primitives. By embedding each of the 8 unique attack variants into the 10 foundation models, we generated a comprehensive corpus of 80 advanced malicious models across different domains.

For all datasets, we treat the model artifact as the unit under test. Labels are assigned by manual inspection of the payload code and by verifying whether a model performs any malicious operation in a sandboxed environment.

Baselines. We select four malicious model detection methods as baselines. These include three widely adopted third-party detection tools—JFrog [**?**], Protect AI [**?**], and ClamAV [**?**]—which represent the state-of-the-art security scanners currently integrated into the Hugging Face platform, alongside the specialized detection tool provided by the TensorAbuse [**?**] framework.

Crucially, for JFrog, Protect AI, and ClamAV, we directly uploaded our constructed model artifacts to the Hugging Face platform. This methodology triggers the platform's live, integrated security scanning pipelines in real time. By doing so, we capture the authentic detection efficacy and assess the actual defensive posture within a real-world AI model supply chain. For the TensorAbuse detection tool, which focuses on statically identifying suspicious API calls embedded within computational graphs, we utilize the open-source implementation provided by the authors. Since these baseline methods primarily rely on static analysis and signature matching without requiring dynamic execution traces or cross-layer alignment, we evaluate their performance directly against the raw model artifacts under these authentic, platform-native conditions to ensure a rigorous and realistic comparison.

Implementation Details. MAD is implemented in Python. The kernel-level module uses `strace` to collect system calls from the target process and its children. We aggregate raw system-call streams into per-resource operation vectors $\mathbf{v}_{dir}$ and $\mathbf{v}_{sys}$ as described in Section III, and train an Autoencoder on operation vectors extracted from a corpus of trusted benign models. The reconstruction error threshold θ is chosen on a validation set to balance detection rate and false positives.

For Python semantic instrumentation, we apply non-invasive monkey patching to framework and standard-library APIs that can indirectly perform security- relevant operations (e.g., file I/O, network, subprocess). Instrumentation records events of the form $\ell_i = \langle PID, TID, FuncName, ArgsSummary \rangle$ with bounded argument summaries. Cross-layer association then aligns abnormal operations with nearby semantic events using the spatio-temporal strategy in Section III, and constructs behavior profiles that are fed to an LLM for final intent reasoning. We use a production-grade LLM with temperature set to zero for deterministic decisions.

All experiments are conducted on a workstation equipped with dual Intel Xeon Silver 4210R CPUs, 128 GB RAM, and two NVIDIA GeForce RTX 3090 GPUs.

B. RQ1: Detection Performance

To answer this research question, we evaluate the detection performance of MAD and the baseline methods across both the in-the-wild malicious model dataset from MalHug and our advanced TensorAbuse synthetic corpus. This comprehensive evaluation demonstrates MAD's effectiveness in identifying a wide spectrum of malicious activities, ranging from known deserialization exploits to sophisticated framework-native attacks.

Performance on In-the-wild Dataset. We first evaluate the methods on the dataset comprising 87 known malicious models . The results are presented in Table II. As shown, MAD successfully detects 78 out of the 87 malicious models. It is crucial to note the nature of the 9 models that MAD "missed" which were originally flagged by MalHug. Detailed manual analysis revealed that these specific models merely exploit the inherent mechanics of insecure serialization formats to execute benign, normal operations. Their apparent purpose is to demonstrate the exploitability of the vulnerability as a proof-of-concept, rather than executing genuinely harmful actions such as data exfiltration, unauthorized network access, or privilege escalation. Because MAD relies on a two-stage screening process that evaluates the actual intent of the dynamic behavior, it correctly categorizes these models as non-malicious, demonstrating its robust resilience against false positives driven purely by static vulnerability signatures. In stark contrast, the TensorAbuse detection tool fails to identify any of these models, as its static graph-based approach cannot capture deserialization exploits that occur before or outside the computational graph instantiation.

TABLE II: Detection performance (Recall/Number of detected malicious models) of MAD and baselines on the in-the-wild MalHug dataset (Total 87 malicious models).

Method	Detected Malicious Models / Total (Recall)
JFrog	- / 87 (-)
Protect AI	- / 87 (-)
ClamAV	- / 87 (-)
TensorAbuse	0 / 87 (0.00%)
MAD	**78** / 87 (**89.66%**)

TABLE III: Number of detected malicious models by MAD and baselines on the Advanced TensorAbuse Synthetic Corpus, categorized by attack type (20 models per type, 80 models total).

Method	Attack Type (Total models)				
	File Exfil.		Code Insertion	Remote Shell	
JFrog	- / 20	- / 20	- / 20	- / 20	- / 80
Protect AI	- / 20	- / 20	- / 20	- / 20	- / 80
ClamAV	- / 20	- / 20	- / 20	- / 20	- / 80
TensorAbuse	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80
MAD	**20** / 20	**20** / 20	**20** / 20	**20** / 20	**80** / 80

Performance on Advanced Synthetic Corpus. We further evaluate the methods on our rigorously constructed dataset of 80 advanced malicious models, where attacks are deeply embedded within the models' computational graphs. Table III provides a detailed breakdown of the detection results across the four distinct attack categories (File Leakage, IP Exposure, Malicious Code Insertion, and Remote Shell Acquisition). Both MAD and the TensorAbuse detection tool successfully detect 100% of the malicious models in this corpus. The TensorAbuse tool achieves perfect detection here because these specific attacks were generated using its own primitive methodologies, making them highly visible to its static API-checking mechanism. However, as demonstrated in Table II, its static approach fails entirely against attacks outside its predefined scope.

Conversely, MAD achieves this 100% detection rate without relying on static signatures or predefined API lists. By statistically profiling the benign baseline behaviors of models like BERT and VGG16, and dynamically intercepting the anomalous system calls generated by the malicious payloads

such as unexpected network connections or file reads, MAD accurately identifies the malicious intent regardless of the specific attack vector employed. The performance of the remaining baseline tools—JFrog, Protect AI, and ClamAV—varies significantly across the different attack categories, highlighting the limitations of traditional scanning tools when faced with sophisticated, framework-native manipulations.

Overall, the results across both datasets demonstrate that MAD provides a robust, highly generalizable defense mechanism capable of accurately detecting malicious AI model artifacts, regardless of whether they employ common deserialization flaws or advanced, stealthy computational graph manipulations.

C. RQ2: Benign Model Distinction.

While achieving a high detection rate is crucial, a practical defense system must not disrupt normal AI operations by aggressively flagging legitimate models. To answer this research question, we evaluate the false positive rates of MAD and the baseline methods against a rigorously constructed dataset of benign models.

TABLE IV: False Positive evaluation of MAD and baselines on 40 benign models utilizing sensitive TensorFlow APIs. (Lower is better.)

Method	False Positives / Total Benign (FPR)
JFrog	- / 40 (-)
Protect AI	- / 40 (-)
ClamAV	- / 40 (-)
TensorAbuse	40 / 40 (100.00%)
MAD	**0** / 40 (**0.00%**)

Dataset Construction and Justification. To rigorously stress-test the systems, we constructed a dataset of 40 entirely benign models. Crucially, these models were explicitly designed to incorporate the specific TensorFlow APIs such as `tf.raw_ops.ReadFile` or `tf.io.matching_files` that were identified as potential attack vectors in the TensorAbuse methodology. However, rather than weaponizing these APIs, we utilized them strictly for their intended, legitimate purposes—such as reading local dataset configurations, logging debug identities, or performing normal tensor persistence. It is important to emphasize that these are officially supported APIs provided by the TensorFlow standard library. Consequently, developers frequently rely on them in real-world scenarios for complex model training and inference pipelines. Thus, our dataset provides a highly realistic benchmark for evaluating whether a detection tool can understand the context of an API call rather than merely reacting to its presence.

Performance Analysis. The false positive results are presented in Table IV. The findings starkly illustrate the limitations of static, knowledge-dependent detection mechanisms. The TensorAbuse detection tool misclassifies all 40 benign models as malicious, resulting in a 100% False Positive Rate (FPR). Because its detection logic inherently relies on statically scanning the computational graph for a predefined blacklist of "dangerous" APIs, it completely lacks the semantic awareness to differentiate between a legitimate data load and a malicious file exfiltration. Similarly, the traditional scanning tools—JFrog, Protect AI, and ClamAV—misclassify a portion of these models (-/-), further demonstrating that signature-based methods struggle when benign operations overlap with known attack heuristics.

In contrast, MAD successfully identifies all 40 models as benign, achieving a perfect 0.00% FPR. By dynamically observing the runtime execution, MAD's first stage recognizes that the system calls generated by these APIs align with the established statistical baseline of normal framework operations. Even if a specific operation triggers a cross-layer analysis, the LLM-based reasoning engine in the second stage successfully interprets the high-level Python semantics to validate the benign intent. This demonstrates that MAD successfully bridges the semantic gap, providing an exceptionally accurate defense mechanism that secures the supply chain without impeding legitimate developer workflows.

TABLE V: Runtime performance overhead and detection latency of MAD evaluated on standard foundation models.

Model	Loading Time (s)		Inference Throughput		
	Vanilla	w/ MAD	Vanilla		
BERT	-	- (- %)	-	- (- %)	-
CLIP	-	- (- %)	-	- (- %)	-
EfficientNet	-	- (- %)	-	- (- %)	-
VGG16	-	- (- %)	-	- (- %)	-
Average	-	**- (- %)**	-	**- (- %)**	**-**

D. RQ3: Runtime Overhead and Deployability

To evaluate the practical deployability of MAD, we investigate its impact on the host system's performance and its responsiveness to threats. It is important to note that we do not compare MAD with the baseline methods (JFrog, Protect AI, ClamAV, and TensorAbuse) in this evaluation. The baseline methods are strictly static analysis tools designed to scan model artifacts pre-deployment, whereas MAD is a dynamic, runtime defense mechanism. Comparing the one-time scanning duration of static tools against the continuous runtime overhead of a dynamic monitor is fundamentally inequitable. Therefore, our evaluation focuses on the absolute performance impact of MAD on standard model execution.

- *Performance Overhead:* The degradation in model loading time and inference throughput (Samples/Second) caused by deploying MAD.
- *Detection Latency:* The time elapsed from the exact moment a malicious operation is initiated by the model to the generation of the final alert (including Autoencoder screening and Large language model (llm) reasoning).

Performance Overhead Analysis. The performance impact of MAD on normal model execution is presented in Table V.

Fig. 2: The comparison of detection efficiency and token consumption among MAD variants.

The results demonstrate that MAD introduces negligible overhead to both model loading and inference phases. Specifically, the loading time increases by an average of only - %, while the inference throughput degrades by a mere - %.

This near-zero performance penalty is attributed to our architectural design. First, the application-layer dynamic instrumentation employs a highly lightweight boundary-hooking mechanism. Instead of deeply traversing and serializing complex, multi-gigabyte Tensor objects, it only captures high-level metadata (e.g., shapes and types). More importantly, the core analytical engines of MAD—the Autoencoder screening module and the llm reasoning module—run entirely asynchronously on the CPU (or an isolated control plane). Because they do not compete for GPU memory or compute cycles, the intensive matrix multiplications and data parallelism required for AI inference remain completely unhindered.

Detection Latency Analysis. As shown in Table V, MAD achieves an average detection latency of - seconds. This latency encompasses the full pipeline: capturing the low-level system calls, completing the Autoencoder forward pass, extracting the corresponding Python contextual logs, and generating the final llm intent classification. llm intent classification is only invoked on a small fraction of operations marked as abnormal by the Autoencoder, so its cost does not scale with total runtime but with the number of suspicious events.

While dynamic reasoning inherently introduces some delay compared to simple signature matching, this two-stage design keeps the overall overhead within the budget acceptable for offline model vetting pipelines and pre-deployment security checks in practice. Our experiments suggest that MAD can be integrated as a final gate in model release workflows without significantly delaying deployment.

E. RQ4: Component Ablation and Sensitivity

In this section, we evaluate the contribution of each core component of MAD through ablation studies. To demonstrate the necessity of our two-stage cross-layer architecture, we evaluate two variants of MAD:

- *MAD–S*: which removes the unsupervised Autoencoder screening stage, directly feeding all raw system call windows and application-level instrumentation logs to the llm for anomaly detection;
- *MAD–L*: which removes the application-level dynamic instrumentation, relying solely on the anomalous system call windows filtered by the Autoencoder for the llm to make its final judgment without high-level Python semantic context;

We evaluate the performance of these two variants and compare them with the full MAD model. The evaluation metrics cover detection accuracy, including precision, recall, and F1-score, as well as system efficiency involving detection latency and LLM token cost. Table VI details the comprehensive metrics, while Figure 2 visually illustrates the stark differences in system overhead.

TABLE VI: Ablation study of MAD on detection performance, efficiency, and llm token overhead. Latency represents the average detection time per model, and Token Cost represents the average number of tokens consumed per model.

Method	Precision	Recall	F1	Latency (s)	Token Cost
MAD-S	-	-	-	-	-
MAD-L	-	-	-	-	-
MAD	**-**	**-**	**-**	**-**	**-**

Overall, the results show that both variants of MAD exhibit significantly degraded performance or unacceptable system overhead compared to the full model, proving that both stages are indispensable.

Specifically, MAD-S suffers from a catastrophic decrease in detection efficiency and a massive surge in llm token consumption. Because it bypasses the Autoencoder screening, the llm is overwhelmed by the massive volume of benign, repetitive system events generated during normal model loading and inference. This not only increases the average detection latency to an impractical level ($-$ seconds) and incurs extreme API costs ($-$ tokens), but also slightly reduces precision, as the llm is more prone to hallucination when processing excessive background noise. This demonstrates that the statistical screening stage is crucial for filtering out normal activity and maintaining the practical deployability of the system.

On the other hand, MAD-L performs the worst in terms of detection accuracy, particularly suffering a severe drop in Precision ($-$ lower than the full model). By removing the application-level instrumentation logs, the llm is forced to judge intent based entirely on low-level system calls. Without the high-level semantic context such as function arguments to explain why a system call was executed, benign operations are frequently misclassified as malicious network exfiltration. For example, normal distributed synchronization operations may be misjudged as file leakage threats. This confirms our prior observation that a severe semantic gap exists at the OS level, and dynamic cross-layer context alignment is the most important component for eliminating false positives and accurately discerning the true intent of AI models.

REFERENCES

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.